

스마트해운물류 · RailTwin

레일 이상을 어떻게 개발했나

기술 스택, 고른 이유, 그리고 만들던 순서

갠트리 크레인 하부 주행 레일의 이상 구간을
주행 중 센서로 찾아 웹과 3D 레일에 표시하는 디지털 트윈

친구에게 개발 과정을 설명할 때 쓰는 자료입니다.
공식 제출 보고서가 아닙니다.

데이터가 화면까지 가는 길 · 역할을 나눈 방식 · 부품과 서버를 고른 이유 · 코드에서 볼 곳

1. 무엇을 만들고 있나

대형 갠트리 크레인 은 부두에서 컨테이너를 옮기며 바닥의 긴 레일 위를 달립니다. 이 프로젝트는 그 하부 주행 레일에 단차, 국부 침하, 뒤틀림 같은 기하 이상이 생긴 구간을, 크레인이 달리는 동안 센서로 찾아 화면에 찍는 안전관리 시스템입니다.

1차에서 보여주는 것은 결함의 이름보다 어디가 이상한지입니다. 단차인지 침하인지를 세분하는 일은 뒤로 미뤄 두었습니다. 화면은 웹 대시보드와 Unity로 만든 레일 두 줄입니다. 크레인 전체 3D 모델은 지금 범위에 넣지 않았습니다.

한 줄로 말하면, 움직이는 차에서 간격·기울기·위치를 모아 서버가 이상 구간을 정하고, 웹과 3D 레일이 같은 자리를 색으로 보여 줍니다.

데이터가 한 바퀴 도는 길

1	2	3	4	5
좌·우 보드 간격·기울기·위치	MQTT Wi-Fi로 전달	FastAPI 검사 후 규칙 판정	저장·푸시 InfluxDB + WebSocket	화면 웹 + Unity 레일

왼쪽 레일과 오른쪽 레일은 따로 계측합니다. 보드 이름은 rail-left-01, rail-right-01입니다. 화면에서도 left와 right로 나뉩니다.

2. 개발을 어떻게 나눴나

하드웨어, 서버, 화면이 서로를 기다리며 멈추지 않게, 먼저 메시지 형식을 고정했습니다. 보드가 보내는 JSON과 화면이 받는 JSON이 정해지면, 실물 센서가 없어도 서버와 화면은 같은 형식으로 개발할 수 있습니다. 그 자리를 메우는 것이 시연용 더미 스트리머입니다. 약 1m 레일의 40~50cm 구간에서 간격과 롤이 어긋나는 상황을 흉내 냅니다. 이 신호는 연결이 맞는지 보는 용도이고, 실제 레일 성능 수치는 아닙니다.

영역	담당	하는 일
센서·엣지	배용진	보드에서 가속도, 간격, 위치를 모아 Wi-Fi로 보냅니다. 교정과 축 방향도 이쪽입니다.
백엔드·규칙	배준호	받은 값을 검사하고, 간격과 롤로 이상 구간을 정한 뒤 저장하고 화면에 푸시합니다.
웹·Unity	김병서	대시보드와 레일 3D가 같은 좌표, 같은 구간 색을 쓰게 붙입니다.
조립·시험	김정우	축소 모형과 구동부를 맞추고, 반복 주행으로 실측을 채울 차례입니다.

공통으로 맞춘 키는 defects(이상 구간 목록), from_mm-to_mm(구간), distance_x(화면 위치), distance_delta_mm(간격 편차), roll_deg(롤)입니다.

3. 만들던 순서

처음부터 지금 구성이 정해져 있던 것은 아닙니다. 친구에게 설명할 때는 아래 순서가 가장 비슷합니다.

문제를 레일 기하로 좁힘

크레인에서 흔히 말하는 윗부분 처짐이 아니라, 바닥 레일이 울퉁불퉁한지를 보기로 했습니다. 주행 중에 재야 하므로 센서를 레일 전체에 깔기보다, 구동부에 붙여 지나가며 읽습니다.

짧은 수집 시험 뒤 센서를 바꿈

처음에는 ESP32-S3, 가속도 ADXL345, 초음파 HC-SR04로 데이터가 오는지부터 봤습니다. 대상이 강철 레일의 1~8mm 간격이라, 초음파 대신 금속에 반응하는 유도형 근접 센서 GTRIC LR18-08U로 옮겼습니다. 보드는 구동부 안에 넣기 좋은 ESP32-C3 Mini로, 관성 센서는 기울기(롤)까지 보는 MPU-6050으로 바꿨습니다. 그 초기 코드는 firmware/crane_sensor/에 이력으로만 남아 있습니다.

계약을 먼저 적고, 가짜 데이터로 화면까지 이음

보드가 보낼 필드(장치 이름, 위치, 간격 전압, 가속도, 자이로)를 문서로 고정된 뒤 FastAPI가 그 JSON을 구독하게 했습니다. 센서가 꺼져 있어도 DEMO_MODE=true면 같은 WebSocket으로 대시보드와 Unity가 움직입니다. 그래서 펌웨어, 서버, 화면을 병렬로 진행할 수 있었습니다.

판정은 규칙으로 단순화

한동안은 진동 특징을 뽑아 신경망(RBF)으로 위험 정도를 연속값으로 추정하는 길을 코드에 넣어 두었습니다. 2026-09-16에 1차 목표를 바꿨습니다. 실측으로 라벨을 붙인 주행이 아직 없고, 지금 필요한 답은 “이 구간이 이상하다”입니다. 그래서 서버는 LR18 간격이 기준선에서 벗어나고 MPU 롤도 같이 어긋난 구간만 defects로 올립니다. 웨이블릿이나 파고율 특징은 현재 판정에 쓰지 않습니다. 예전 학습 스크립트는 저장소에 남아 있어도 지금 경로가 아닙니다.

3D는 레일만, 브라우저 안에

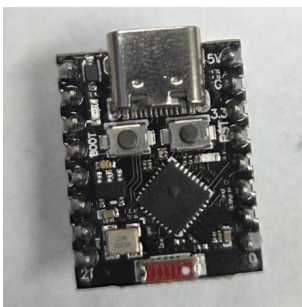
화면 엔진을 검토하는 동안 Godot으로 위치를 받아 보던 시기가 있습니다. 2차 평가에 Unity를 넣기로 하면서, WebGL로 빌드해 대시보드 페이지 안에 레일만 띄웁니다. 구매한 상용 크레인 모델과, 실제로 만든 평판 모형은 목적과 크기가 달라 같은 모델로 다루지 않습니다.

4. 엣지와 센서 — 왜 이 부품인가

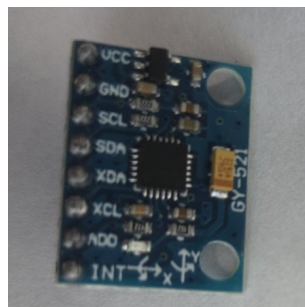
차체 안에 넣는 작은 노드가 금속과의 간격, 차체가 기운 정도, 지금 몇 mm를 달렸는지를 같이 읽어야 합니다.

구성	하는 일	이렇게 고른 이유
ESP32-C3 Mini × 2	수집과 Wi-Fi 송신	좌우 레일이 따로라 노드를 둘로 나눴습니다. Wi-Fi가 붙어 있어 주행 중 선 없이 서버로 보냅니다.
MPU-6050	가속도·자이로 100Hz	간격만으로는 뒤틀림이 잘 안 보입니다. 6축이라 롤(기울기)을 한 칩에서 추정합니다.
LR18-08U	금속까지 간격 1~8mm	유도형은 강재 표면의 간격 변화를 연속 전압(0~10V)으로 냅니다. 동작 전원은 DC 15~30V입니다.
ADS1115	그 전압을 16bit	간격 전압을 안정적으로 읽으려고 외부 ADC를 씁니다. 0~10V는 입력 범위를 넘으므로 분압·보호를 거칩니다. 공칭 20Hz입니다.
엔코더	주행 위치	이상 값을 시각이 아니라 레일 좌표에 찍으려면 같은 순간의 위치가 필요합니다. 바퀴 지름 기준은 53mm입니다.
L298N, 기어드 모터	모형이 직접 주행	서 있는 센서가 아니라 지나가며 재야 해서, 축소 모형이 레일 위를 달립니다.
18650, MT3608	보드·센서 전원	유도형 센서는 15V 이상이 필요해서 배터리를 승압합니다. 센서 전원은 보드 3.3V와 분리합니다.

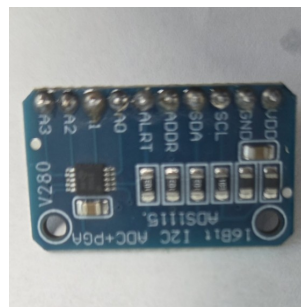
펌웨어는 Arduino C++입니다. Adafruit 센서 라이브러리, ArduinoJson, MQTT 클라이언트를 씁니다. IMU 샘플 10개를 JSON 한 묶음으로 보내 업로드는 대략 10Hz입니다. 정지 중 미세 흔들림은 보드에서 걸러 불필요한 전송을 줄입니다. USB 시리얼은 진단용이고, 실제 데이터 길은 Wi-Fi입니다.



ESP32-C3 Mini



MPU-6050



ADS1115



GTRIC LR18-08U

실물로 확인한 현재 센서 모듈의 주요 부품입니다.

5. 전송과 서버 — 왜 이렇게 이었나

구성	하는 일	이렇게 고른 이유
MQTT QoS 1 Mosquitto	보드 → 서버	좌우 노드가 각자 발행하고 서버는 구독만 합니다. QoS 1은 끊겼다 붙어도 한 번은 도착하게 하고, 상태 토픽으로 노드가 살아 있는지도 구분합니다.
WebSocket /ws	서버 → 화면	브라우저와 Unity는 MQTT 클라이언트가 아닙니다. 같은 JSON을 밀어 주면 대시보드와 3D가 한 스트림을 같이 봅니다.
Python, FastAPI, Uvicorn	한 프로세스의 API	수신, 판정, 저장, 화면 푸시를 비동기로 묶기 쉽습니다. HTTP 상태(/health)와 WebSocket이 같이 있습니다.
asyncio 큐	수신과 판정 분리	MQTT 콜백이 계산에 막히지 않게 합니다. 큐가 차면 오래된 샘플을 버려 최신 위치를 우선합니다. 파싱 실패와 DB 실패는 서로 끊습니다.
InfluxDB	시계열 저장	주행 기록은 시간 축입니다. 실시간 화면과 나중 조회를 나눕니다. DB가 비어 있어도 화면 푸시는 동작합니다.
간격+롤 규칙	이상 구간	두 신호가 같이 어긋난 구간만 표시합니다. 종류 이름은 비워 둡니다. 라벨된 실측이 쌓이기 전에는 이 규칙이 설명하기 쉽습니다.
더미 스트리머	하드웨어 없이 시연	담당 영역을 병렬로 붙이고, 발표 때 보드가 없어도 파이프라인이 도는지 확인합니다.

토픽은 rail/v1/nodes/{장치이름}/telemetry 와 status 입니다. FastAPI 계정은 구독 전용입니다. 운영에서는 TLS(MQTTS)를 목표로 두었고, 로컬 묶음은 Docker Compose로 Mosquitto와 API를 같이 올립니다.

6. 화면과 배포

구성	하는 일	이렇게 고른 이유
HTML, JS, Tailwind	한 페이지 대시보드	별도 프론트 빌드 없이 서버가 페이지를 엽니다. 센서값, 간격 편차, 롤, 이상 구간, 연결 상태를 보여 줍니다.
Unity 6 WebGL	레일 두 줄	평가 범위에 Unity가 있습니다. WebGL이라 대시보드 안에 넣습니다. 구간 색은 정상에서 이상으로 파랑-초록-노랑-빨강입니다.
distance_x, rail_risk	위치와 색의 공용 키	엔코더 mm를 cm로 바꿔 웹과 Unity가 같은 자리를 봅니다. 지나간 구간의 위험도는 최댓값을 남겨 한 번 찍힌 구간이 바로 사라지지 않게 합니다.
Docker Compose	서버를 같은 방식으로	API와 MQTT 브로커를 한 번에 띄우고, 죽으면 다시 올립니다.
네이버 클라우드	밖에 보여주는 주소	시연용으로 대시보드와 MQTT 포트를 열었습니다. HTTPS 페이지는 암호화되지 않은 WebSocket을 막으므로 화면은 WSS를 씁니다.
GitHub, Notion	코드와 일정	코드 이력은 GitHub, 작업 기록은 Notion입니다. 배포 워크플로도 GitHub Actions에 있습니다.

7. 친구에게 이렇게 말하면 됩니다

“크레인이 레일을 달릴 때, 양쪽에 붙인 작은 보드가 금속과의 간격이랑 기울기, 지금 위치를 무선으로 보내. 서버는 그 두 신호가 같이 이상한 구간만 골라서 저장하고, 웹이랑 3D 레일에 바로 칠해. 인공지능으로 결함 이름을 맞추기 전에, 어디를 봐야 하는지를 먼저 찍는 단계야.”

코드로 파이프라인이 도는 것과, 반복 주행으로 실측을 저장한 것은 다릅니다. 2026-08-23 확인 기준으로 펌웨어 동작, 구동부, 모형 조립은 끝났고, 반복 주행 시험과 실측 저장은 아직입니다. 더미로 보인 화면을 현장 정확도처럼 말하면 안 됩니다.

8. 저장소에서 어디를 보면 되나

경로	무엇인가	열어 보면
firmware/esp32_c3_rail_sensor/	현재 펌웨어	좌우 프로파일, MQTT 발행, 센서 읽기.
main.py	서버	MQTT 구독, 큐, 규칙 판정, InfluxDB, WebSocket, 더미.
rail_defect.py	판정 규칙	간격 편차와 롤로 defects를 만듭니다.
frontend/index.html	대시보드	표와 레일 뷰, Unity가 없을 때의 브라우저 레일.
unity/RailTwinRails/	Unity 프로젝트	레일만 있는 WebGL 씬.
docs/esp32-mqtt-contract.md	데이터 계약	보드가 보내는 필드 설명.
README.md	실행 순서	환경 파일, 서버 실행, 대시보드 주소.

로컬에서 화면만 보려면 .env.example을 .env로 복사한 뒤 DEMO_MODE=true로 두고 `uvicorn main:app --host 0.0.0.0 --port 8000` 을 실행하면 됩니다. 대시보드는 서버의 /dashboard 입니다.

이 글은 2026-09-16 이후의 목표 스택(간격+롤 규칙, Unity WebGL 레일)을 기준으로 정리했습니다. 개발보고서 초안 일부에는 그 이전의 RBF-Godot 설명이 남아 있을 수 있습니다.